
LAPORAN PRAKTIKUM

TEKNOLOGI IOT II



Disusun oleh :

Nama : Salim Suheri Sitompul

NPM 25782053

Kelas : TRI-3B

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

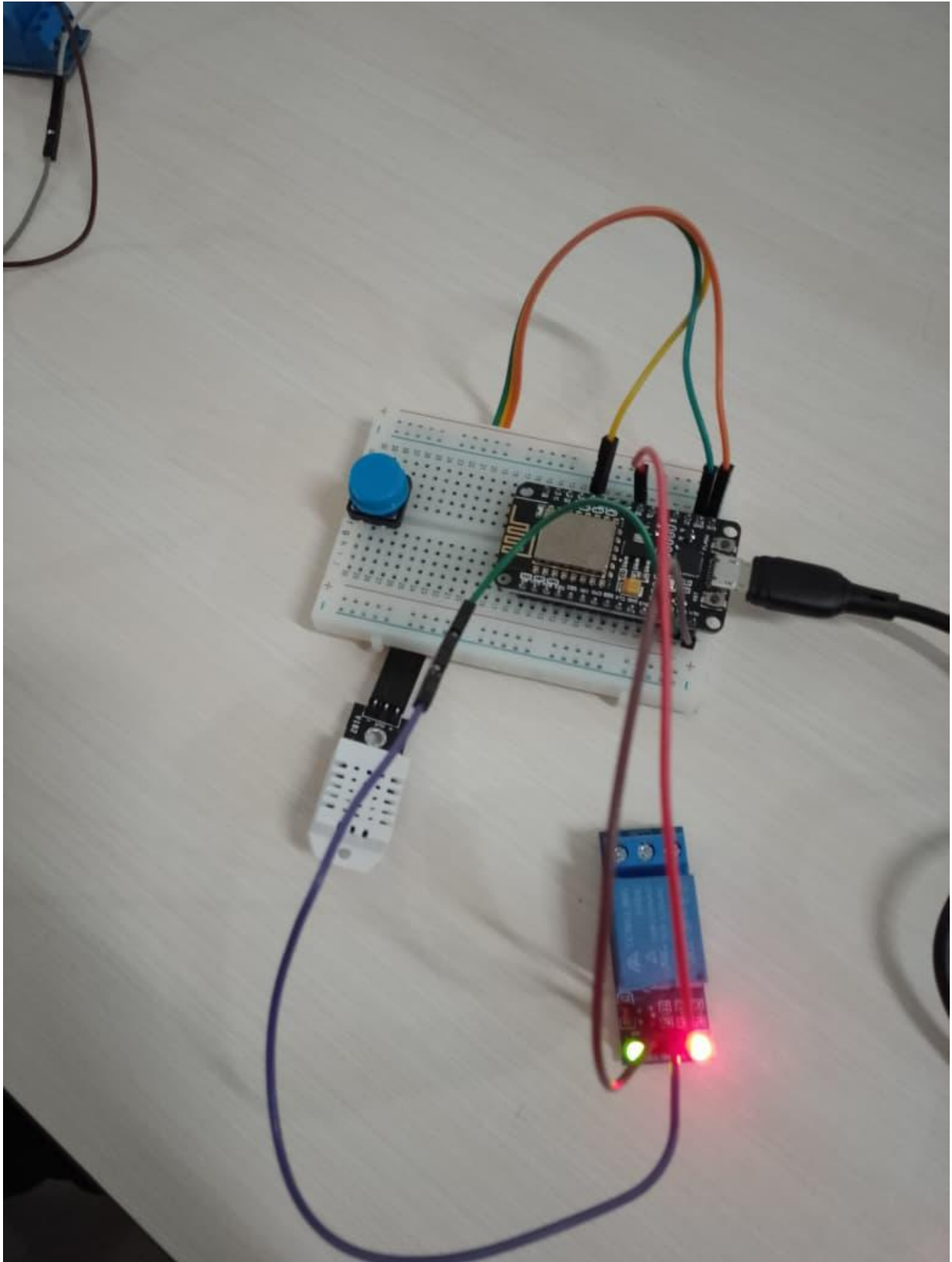
TEKNOLOGI REKAYASA INTERNET

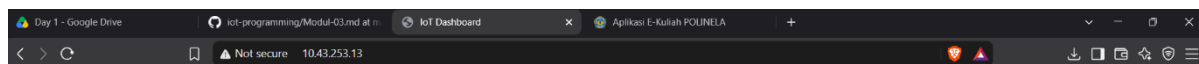
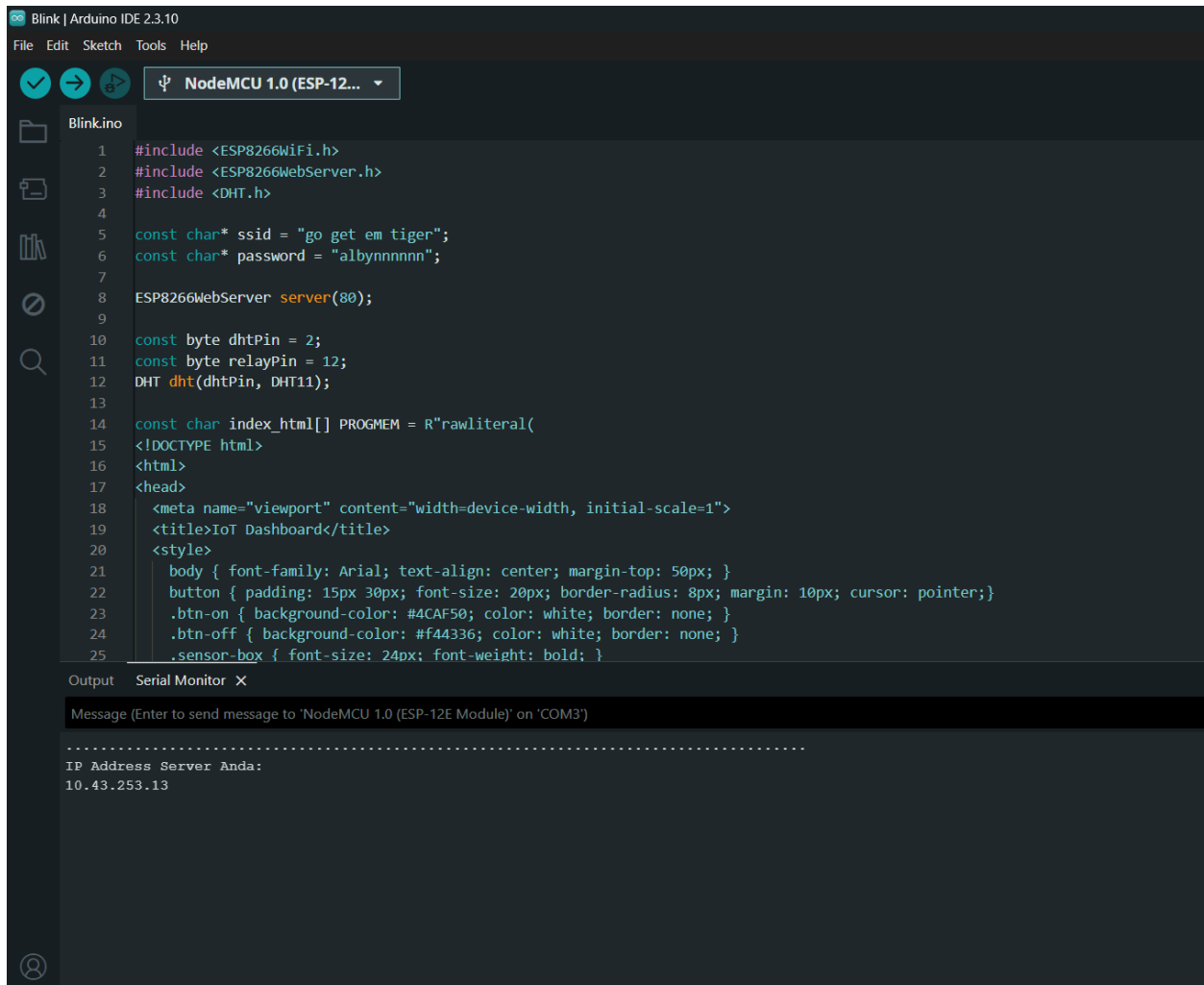
JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

3.5.1 Praktikum 1: NodeMCU sebagai *Web Server* (Menyajikan HTML)





ESP8266 Web Server

Suhu Saat Ini: **1.00** Celcius

Kendali Relay

ON

OFF

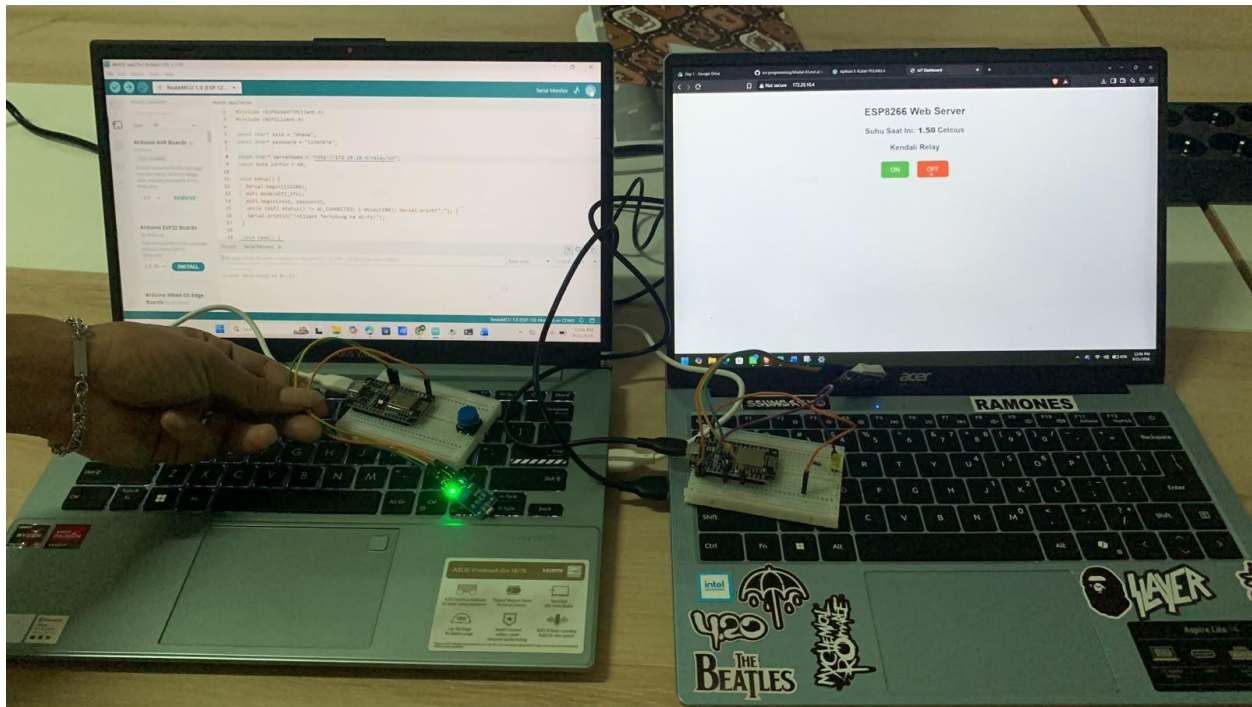
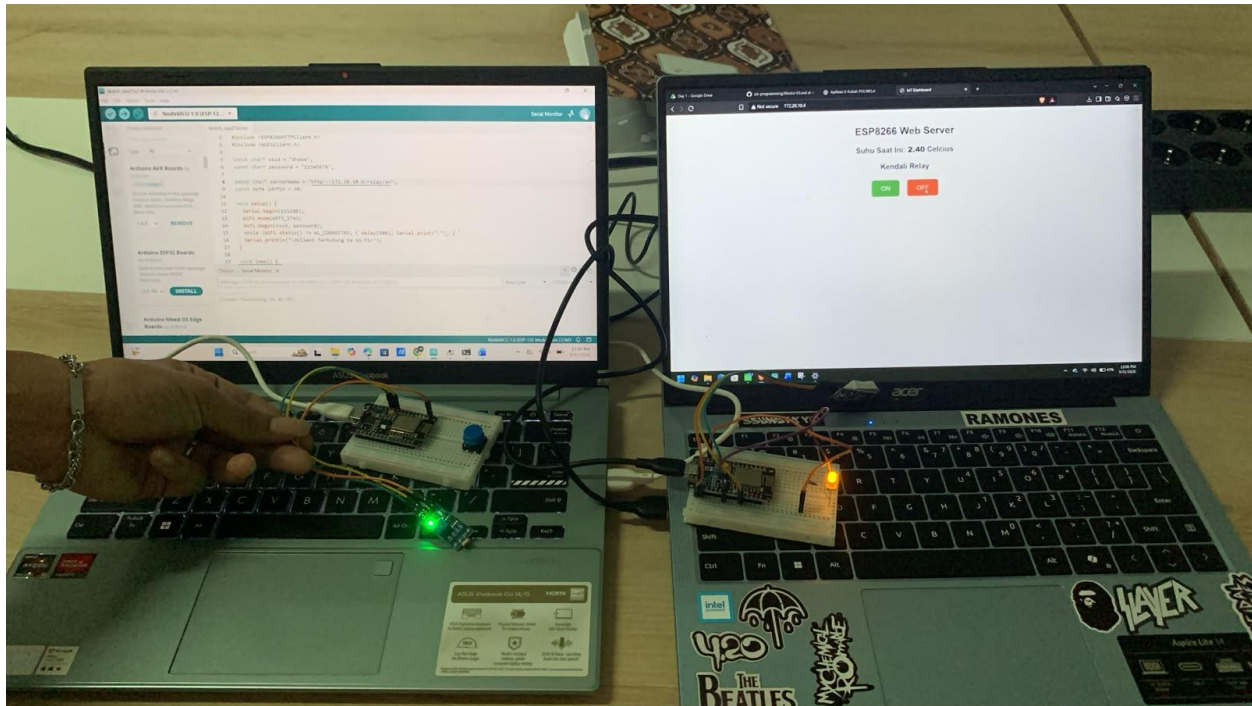
Langkah Kerja Rangkaian:

1. Hubungkan pin DATA / OUT / S DHT ke pin D4 (GPIO 2). (Pastikan VCC DHT ke 3V3, GND ke GND).
2. Hubungkan pin IN / Signal Relay ke pin D6 (GPIO 12). (Pastikan VCC Relay ke VIN/VU NodeMCU, GND ke GND).

Penjelasan Singkat Kode:

- `ESP8266WebServer server(80);`: Menginisialisasi *library Server* agar terus mendengarkan panggilan di jalur lalu lintas standar web (*Port 80*).
- `html.replace("%TEMPERATURE%", String(t));`: Fitur manipulasi teks (*String*). Perintah ini secara cerdas akan menyapu dokumen HTML, mencari teks *placeholder* `%TEMPERATURE%`, dan menggantinya dengan variabel angka suhu yang didapat dari sensor aktual sebelum dikirim ke layar pengunjung. Inilah esensi dari halaman *web* dinamis.
- `server.on("/relay/on", handleRelayOn);`: Fungsi *Routing*. Jika ada klien mengunjungi tautan URL berakhiran `/relay/on`, server akan menjalankan aksi mengubah *state* Relay, lalu me- *redirect* layar pengunjung kembali ke halaman depan (Location: `/`) via kode 303.
- `server.handleClient();`: Berada di dalam `loop()` untuk terus-menerus mengecek apakah ada permintaan pengunjung yang datang setiap milidetiknya.

3.5.2 Praktikum 2: NodeMCU sebagai *HTTP Client* (Meminta Layanan)



```

sketch_sep21a.ino
1  #include <ESP8266WiFi.h>
2  #include <ESP8266HTTPClient.h>
3  #include <WiFiClient.h>
4
5  const char* ssid = "dhewa";
6  const char* password = "12345678";
7
8  const char* serverName = "http://172.20.10.4/relay/on";
9  const byte ldrPin = A0;
10
11 void setup() {
12     Serial.begin(115200);
13     WiFi.mode(WIFI_STA);
14     WiFi.begin(ssid, password);
15     while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
16     Serial.println("\nClient Terhubung ke Wi-Fi!");
17 }
18

```

Langkah Kerja Rangkaian:

1. Rangkai sensor LDR dan Resistor 10k Ohm sebagai pembagi tegangan (seperti di Modul 2) lalu hubungkan jalur sinyalnya ke pin A0.

Penjelasan Singkat Kode:

- WiFiClient client; dan HTTPClient http;; Kelas pemanggil untuk memulai eksekusi permintaan *client* jarak jauh (mirip fungsi aplikasi *browser* di laptop).
- http.begin(client, serverName);: Menargetkan sasaran URL *server*.
- int httpResponseCode = http.GET();: Mengeksekusi ketukan pintu (*Request* HTTP GET). Jika Server merespons sukses, nilai variabel ini akan berisi angka standar jaringan seperti 200 (OK) atau 303 (Redirect). Jika URL rusak atau *Server offline*, variabel akan mereturn nilai negatif.
- delay(10000);: *Cooldown* panjang 10 detik ini ditambahkan agar *Client* tidak memborbardir (*flood/DDoS*) Server dengan permintaan on secara terus-menerus setiap detik ketika kondisinya sedang gelap.

3.6 Latihan

Latihan 1 (Pemahaman Jaringan):

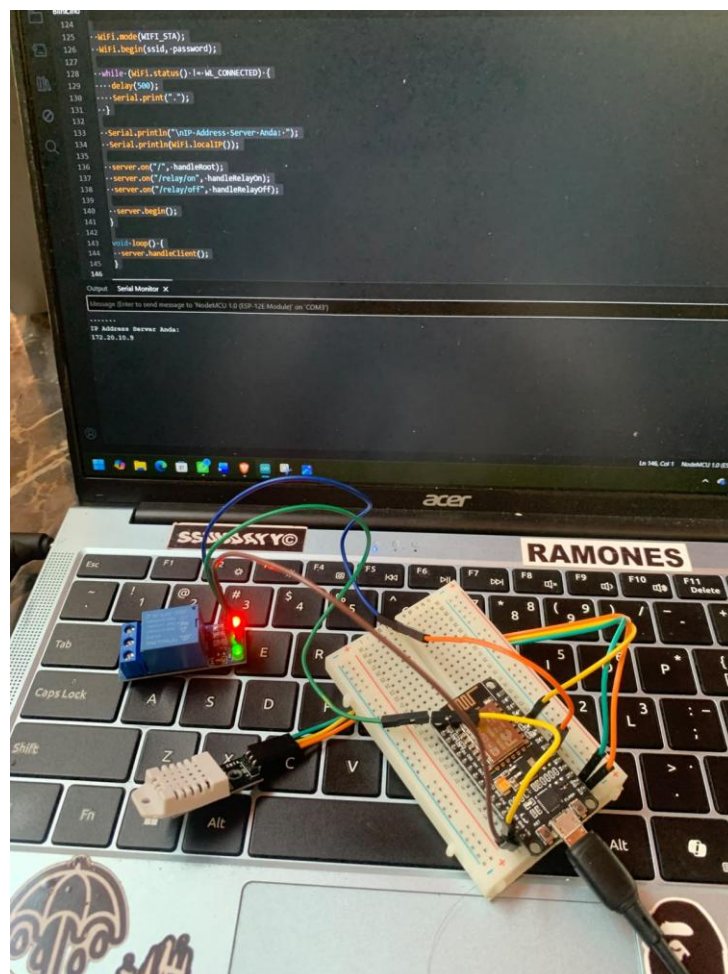
Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan *Station (STA)* pada kedua praktikum di atas, dan bukan mode *Access Point (AP)*?

Jawaban: Mode Station (STA) digunakan karena NodeMCU pada praktikum berperan sebagai perangkat yang terhubung ke jaringan Wi-Fi yang sudah tersedia, misalnya Wi-Fi dari router atau hotspot. Dengan mode STA, NodeMCU dapat memperoleh alamat IP dari jaringan dan berkomunikasi dengan perangkat lain dalam jaringan tersebut.

Sedangkan mode Access Point (AP) tidak digunakan karena AP membuat NodeMCU menjadi pemancar jaringan Wi-Fi sendiri. Pada praktikum ini NodeMCU tidak ditujukan untuk menyediakan jaringan, tetapi untuk terhubung dan berkomunikasi melalui jaringan yang sudah ada.

Latihan 2 (Modifikasi Antarmuka & Telemetry):

Pada Praktikum 1, tampilan web saat ini baru menyajikan data suhu ruangan dan masih menggunakan dua tombol terpisah (ON dan OFF). Lakukan dua modifikasi berikut pada *Web Server*:



ESP8266 Web Server

Suhu Saat Ini: **1.30** Celcius

Kelembapan: **17.30** %

Kendali Relay

NYALAKAN (ON)



ESP8266 Web Server

Suhu Saat Ini: **2.00** Celcius

Kelembapan: **14.50** %

Kendali Relay

MATIKAN (OFF)

- Bagian Kode HTML yang dimodifikasi

```

56     <div class="sensor-box">
57         <p>Suhu Saat Ini: <strong>%TEMPERATURE%</strong> Celcius</p>
58         <p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>
59     </div>
60
61     <h2>Kendali Relay</h2>
62
63     %RELAY_BUTTON%

```

- Fungsi handleRoot() yang dimodifikasi lengkap

```

69 void handleRoot() {
70     String html = index_html;
71
72     float t = dht.readTemperature();
73     float h = dht.readHumidity();
74
75     // Mengganti placeholder suhu
76     if (isnan(t)) {
77         html.replace("%TEMPERATURE%", "--");
78     } else {
79         html.replace("%TEMPERATURE%", String(t));
80     }
81
82     // Mengganti placeholder kelembapan
83     if (isnan(h)) {
84         html.replace("%HUMIDITY%", "--");
85     } else {
86         html.replace("%HUMIDITY%", String(h));
87     }
88
89     // Membuat tombol berdasarkan kondisi relay
90     if (digitalRead(relayPin) == LOW) {
91         html.replace(
92             "%RELAY_BUTTON%",
93             "<a href='/relay/on'><button class='btn-on'>NYALAKAN (ON)</button></a>"
94         );
95     } else {
96         html.replace(
97             "%RELAY_BUTTON%",
98             "<a href='/relay/off'><button class='btn-off'>MATIKAN (OFF)</button></a>"
99         );
100    }
101
102    server.send(200, "text/html", html);
103 }
104

```

Modifikasi Antarmuka dan Telemetri:

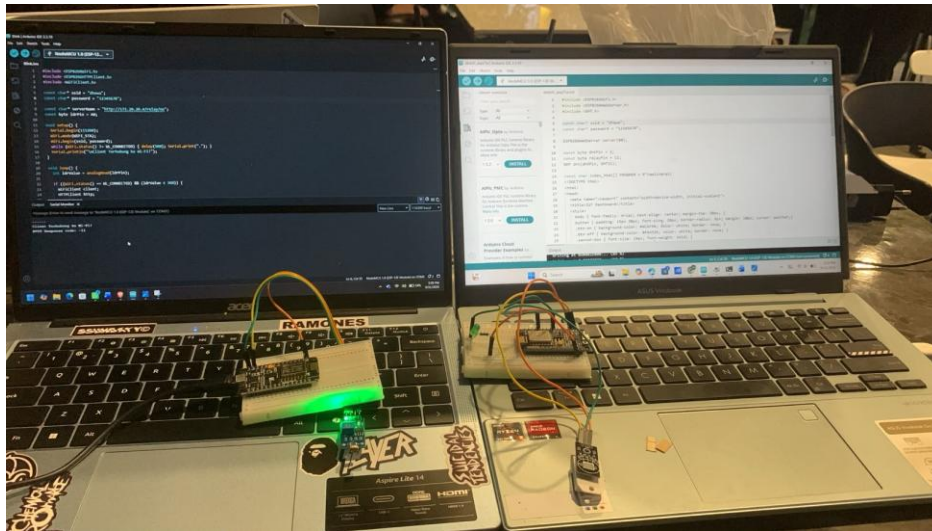
Pada latihan ini dilakukan dua modifikasi pada Web Server ESP8266. Modifikasi pertama adalah menambahkan pembacaan kelembapan menggunakan fungsi `dht.readHumidity()`. Nilai kelembapan ditampilkan pada halaman web dengan menambahkan placeholder `%HUMIDITY%`, kemudian placeholder tersebut diganti dengan nilai kelembapan melalui `html.replace()` pada fungsi `handleRoot()`.

Modifikasi kedua adalah mengubah dua tombol ON dan OFF menjadi satu tombol dinamis (toggle button). Program membaca kondisi relay menggunakan `digitalRead(relayPin)`. Jika relay dalam kondisi OFF (LOW), tombol akan menampilkan "NYALAKAN (ON)" dan mengarahkan ke `/relay/on`. Jika relay dalam kondisi ON (HIGH), tombol akan menampilkan "MATIKAN (OFF)" dan mengarahkan ke `/relay/off`.

Dengan modifikasi tersebut, halaman web dapat menampilkan suhu dan kelembapan, serta menyediakan satu tombol yang secara otomatis menyesuaikan dengan kondisi relay saat ini.

Latihan 3 (Analisis Kode Client):

Pada Praktikum 2, *Serial Monitor Client* mencetak HTTP Response code. Analisislah apa yang akan dicetak oleh *Serial Monitor* Anda jika ternyata NodeMCU Server teman Anda mendadak kehilangan daya (*offline*)? (Silakan cabut daya Server dan lihat hasil erornya).



```
Blink (Arduino IDE 2.3.10)
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...)
Blink.ino
1 #include <ESP8266WiFi.h>
2 #include <ESP8266HTTPClient.h>
3 #include <WiFiClient.h>
4
5 const char* ssid = "dhuwa";
6 const char* password = "12345678";
7
8 const char* serverName = "http://172.20.20.4/relay/on";
9 const byte idrPin = A0;
10
11 void setup() {
12   Serial.begin(115200);
13   WiFi.mode(WIFI_STA);
14   WiFi.begin(ssid, password);
15   while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
16   Serial.println("\nClient Terhubung ke Wi-Fi");
17 }
18
19 void loop() {
20   int idrValue = analogRead(idrPin);
21
22   if ((WiFi.status() == WL_CONNECTED) && (idrValue < 300)) {
23     WiFiClient client;
24     HTTPClient http;
25
26     http.begin(client, serverName);
27     int httpCode = http.GET();
28     Serial.println("HTTP Response code: " + String(httpCode));
29   }
30 }
```

Output Serial Monitor X

Message [Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3']

Client Terhubung ke Wi-Fi

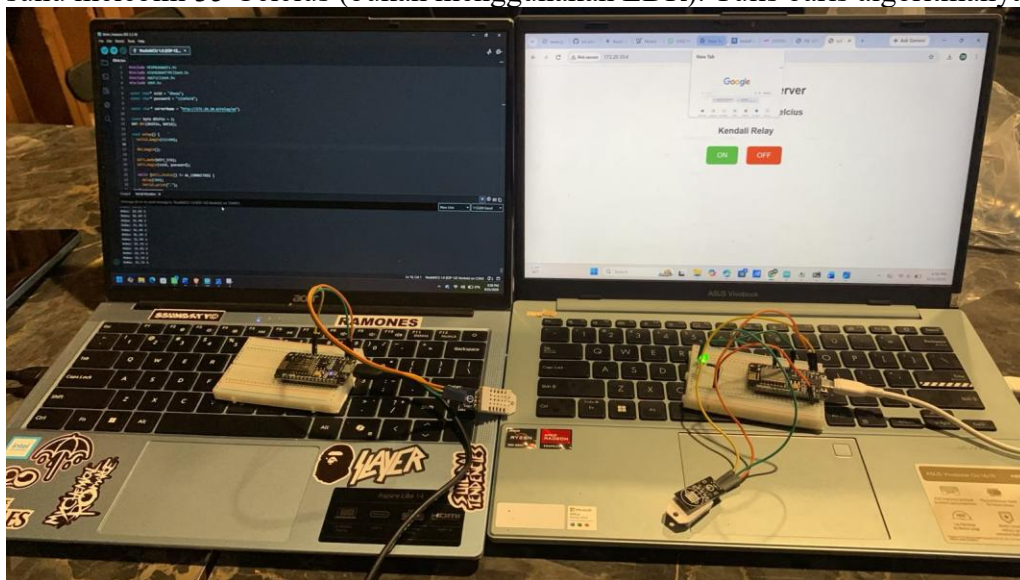
HTTP Response code: -11

Hasil Analisis: Pengujian ini dilakukan dengan menjalankan NodeMCU Client dan NodeMCU

Server untuk memastikan keduanya dapat berkomunikasi. Setelah itu, NodeMCU Server dimatikan dengan cara mencabut kabel USB atau sumber daya yang terhubung ke Server, sedangkan NodeMCU Client tetap menyala. Selanjutnya Client mencoba mengirim GET Request ke Server yang sudah dalam keadaan offline. Berdasarkan hasil pengujian pada Serial Monitor, Client tetap berhasil terhubung ke Wi-Fi dan menampilkan pesan “Client Terhubung ke Wi-Fi!”, tetapi ketika mencoba berkomunikasi dengan Server, muncul “HTTP Response code: -11”. Hal ini menunjukkan bahwa Client tidak mendapatkan respons HTTP dari Server karena Server dalam keadaan offline.

Latihan 4 (Pengembangan Algoritma):

Anda ingin mendesain Klien Pendeteksi Suhu. Modifikasi *conditional block* if di Praktikum 2 sehingga *Client* mengirim permintaan (GET Request) **HANYA JIKA** sensor DHT membaca suhu melebihi 35 Celcius (bukan menggunakan LDR). Tulis baris algoritmanya



```
1 #include <ESP8266WiFi.h>
2 #include <ESP8266HTTPClient.h>
3 #include <WiFiClient.h>
4 #include <DHT.h>
5
6 const char* ssid = "dhewa";
7 const char* password = "12345678";
8
9 const char* serverName = "http://172.20.10.4/relay/on";
10
11 const byte dhtPin = 2;
12 DHT dht(dhtPin, DHT22);
13
14 void setup() {
15     Serial.begin(115200);
16
17     dht.begin();
18
19     WiFi.mode(WIFI_STA);
20     WiFi.begin(ssid, password);
21
22     while (WiFi.status() != WL_CONNECTED) {
23         delay(500);
24         Serial.print(".");
25     }
26 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

```
-----
Suhu: 34.70 C
Suhu: 34.80 C
Suhu: 34.80 C
Suhu: 34.90 C
Suhu: 34.90 C
Suhu: 35.00 C
Suhu: 35.00 C
Suhu: 35.00 C
Suhu: 35.00 C
Suhu: 35.10 C
HTTP Response code: 303
Suhu: 35.00 C
Suhu: 35.00 C
```

Bagian yang di lakukan modifikasi pada kode:

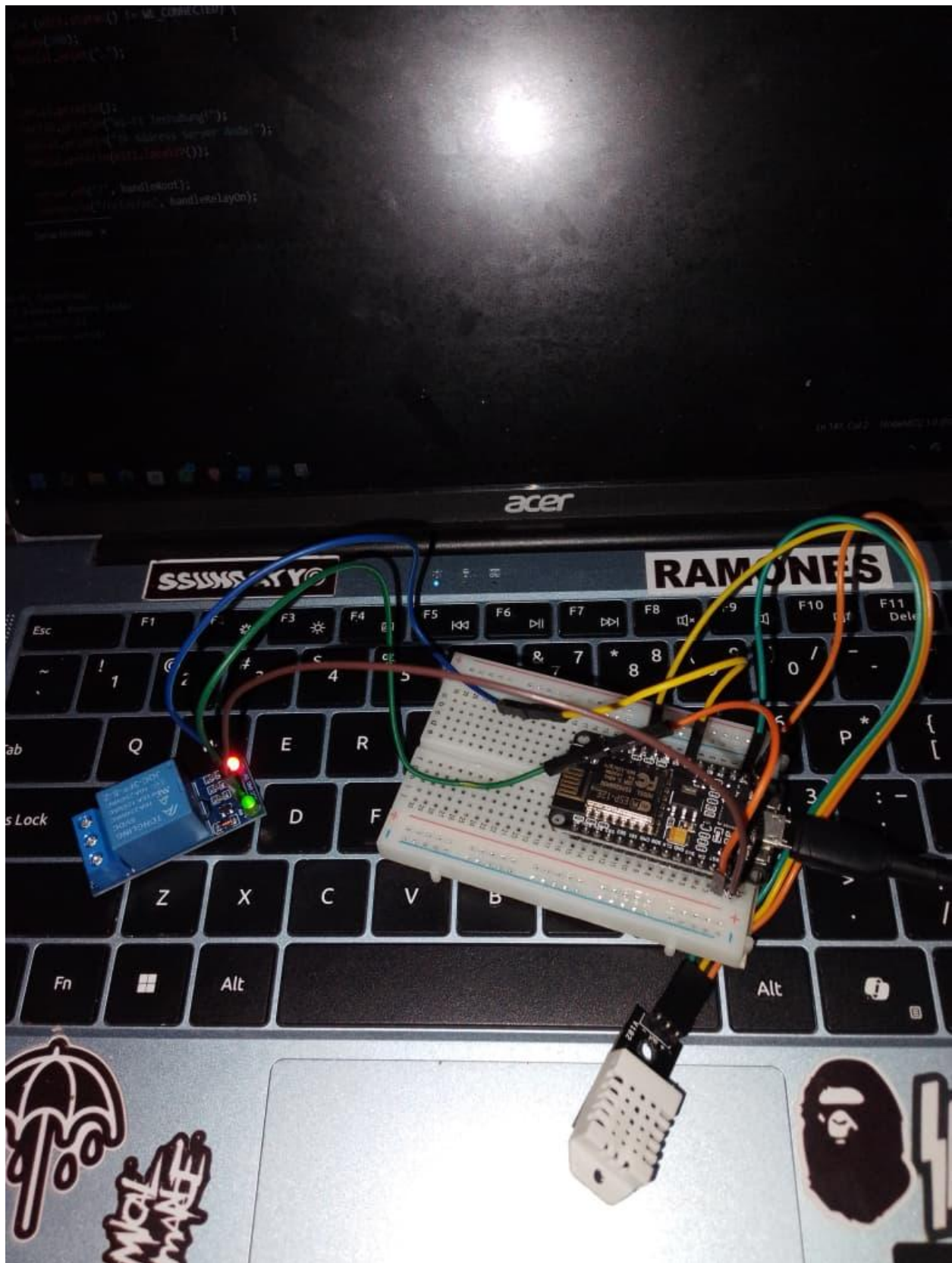
```
4  #include <DHT.h>
5
6  const char* ssid = "dhewa";
7  const char* password = "12345678";
8
9  const char* serverName = "http://172.20.10.4/relay/on";
10
11 const byte dhtPin = 2;
12 DHT dht(dhtPin, DHT22);
13
14 void setup() {
15     Serial.begin(115200);
16
17     dht.begin();
18
19     WiFi.mode(WIFI_STA);
20     WiFi.begin(ssid, password);
21
22     while (WiFi.status() != WL_CONNECTED) {
23         delay(500);
24         Serial.print(".");
25     }
26
27     Serial.println("\nClient Terhubung ke Wi-Fi!");
28 }
29
30 void loop() {
31     float temperature = dht.readTemperature();
32
33     Serial.print("Suhu: ");
34     Serial.print(temperature);
35     Serial.println(" c");
36
37     if ((WiFi.status() == WL_CONNECTED) && (temperature > 35)) {
38         WiFiClient client;
39         HTTPClient http;
40
41         http.begin(client, serverName);
42         int httpStatusCode = http.GET();
43     }
```

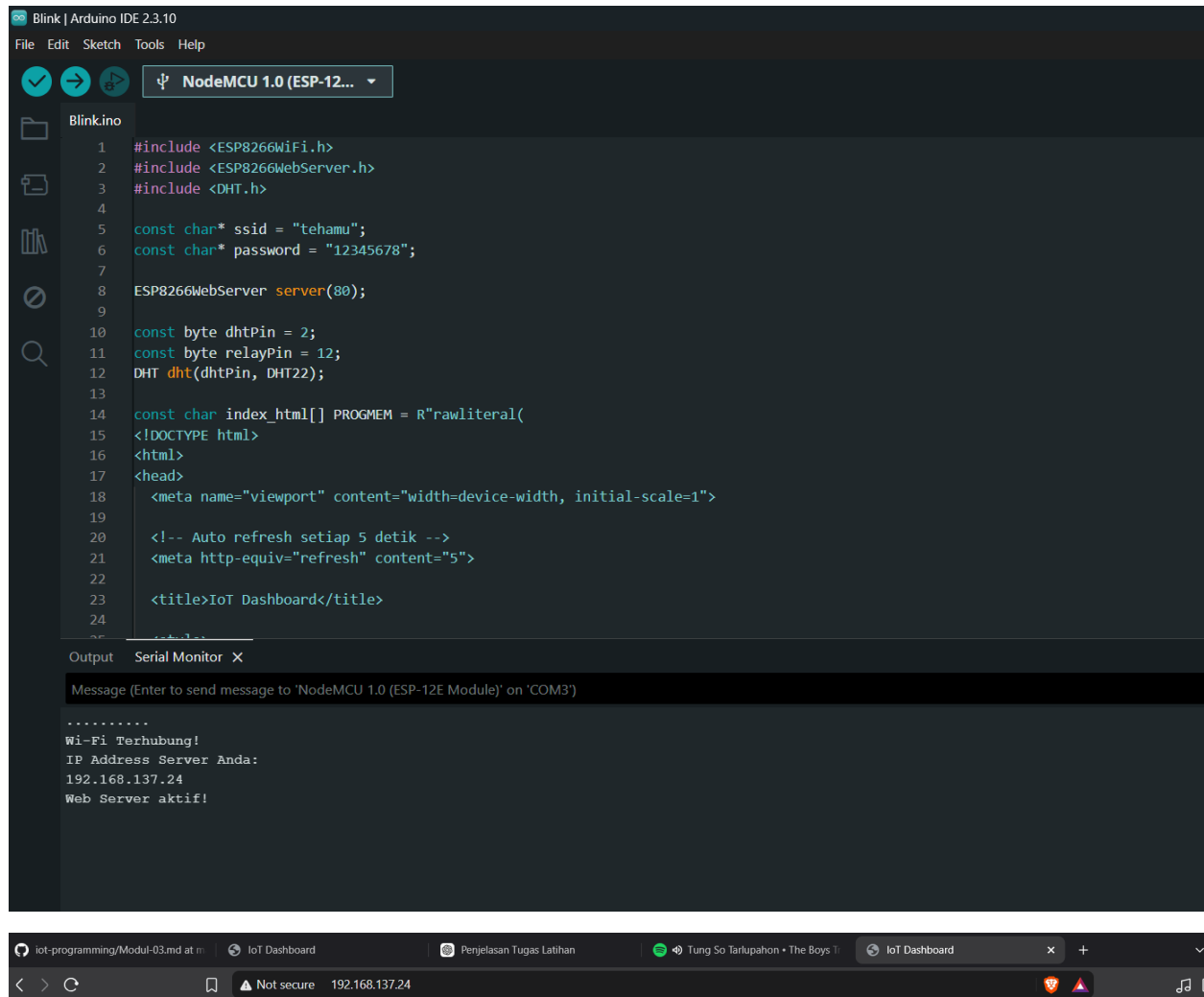
Pada Latihan 4, sensor LDR pada Praktikum 2 diganti dengan sensor DHT22 untuk membaca suhu. Library DHT.h ditambahkan agar NodeMCU dapat menggunakan sensor DHT22. Kemudian dht.readTemperature() digunakan untuk mengambil nilai suhu.

Program selanjutnya menggunakan kondisi: temperature > 35

Artinya, GET Request hanya dikirim ke Server jika suhu lebih dari 35°C. Jika suhu masih 35°C atau kurang, Client tidak mengirim request ke Server. Dengan demikian, pengiriman perintah ke Server sekarang dikendalikan berdasarkan nilai suhu dari sensor DHT22, bukan lagi berdasarkan nilai LDR.

3.7 Tugas: Pengembangan *Web Server* Cerdas dan Analisis Arsitektur





ESP8266 Web Server

Suhu Saat Ini: 32.50 Celcius

Kendali Relay

ON - OFF

Kode yang di modifikasi/di tambahkan

```
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">

  <!-- Auto refresh setiap 5 detik -->
  <meta http-equiv="refresh" content="5">

  <title>IoT Dashboard</title>
```

Berdasarkan hasil pengujian, dashboard berhasil melakukan refresh secara otomatis setiap 5 detik tanpa perlu menekan tombol refresh pada browser. Nilai suhu juga dapat diperbarui setelah halaman melakukan refresh.

Namun, penggunaan **Meta Refresh** memiliki beberapa kelemahan. Saat halaman melakukan refresh, tampilan dashboard terlihat **berkedip** karena seluruh halaman dimuat ulang. Selain itu, setiap 5 detik browser harus meminta kembali seluruh halaman dari ESP8266 sehingga dapat menggunakan bandwidth lebih banyak jika halaman sering diperbarui.

Penggunaan Meta Refresh juga dapat mengganggu interaksi pengguna. Jika pengguna menekan tombol **ON/OFF** tepat ketika halaman sedang melakukan refresh, terdapat kemungkinan interaksi terasa kurang nyaman karena halaman sedang dimuat ulang.

Kesimpulan: Meta Refresh berhasil digunakan untuk memperbarui data suhu secara otomatis, tetapi metode ini masih memiliki kelemahan pada tampilan dan efisiensi komunikasi karena seluruh halaman harus dimuat ulang setiap 5 detik.